□ POC-1: E-Commerce Analytics Platform (PRODUCTION STYLE)

ADLS → Databricks → Snowflake → dbt → Power BI

□ OVERALL FLOW

STEP 1 → Upload raw data to ADLS (Raw Zone)
STEP 2 → Databricks reads ADLS (Bronze)
STEP 3 → Clean + transform (Silver)
STEP 4 → Aggregate (Gold)
STEP 5 → Load Gold → Snowflake
STEP 6 → dbt models (Analytics layer)
STEP 7 → Power BI dashboards

□ PHASE 0 — ENVIRONMENT SETUP (DO FIRST)

0.1 Create Azure Resources

In Azure Free Subscription:

Create:

1. Storage Account (ADLS Gen2)
2. Databricks Workspace (trial/free tier if available)

0.2 Enable ADLS Gen2

When creating storage:

✔ Enable:

- Hierarchical namespace = TRUE

0.3 Create Containers

Inside Storage Account:

raw
bronze
silver
gold

0.4 Snowflake Setup

Create:

```
CREATE DATABASE ECOM_DB;
CREATE SCHEMA RAW;
CREATE SCHEMA MARTS;
CREATE WAREHOUSE ECOM_WH;
```

0.5 dbt Setup (Local)

dbt init ecommerce_project

□ PHASE 1 — RAW DATA CREATION

1.1 Create Local Dataset Folder

data/

1.2 Create Files

customers.csv

products.csv

orders.csv

---

payments.csv

□ PHASE 2 — UPLOAD DATA TO ADLS (RAW ZONE)

2.1 Upload via Azure Storage Explorer

Upload:

customers.csv → raw/customers/
products.csv → raw/products/
orders.csv → raw/orders/
payments.csv → raw/payments/

2.2 Final ADLS Structure

raw/
 customers/
 products/
 orders/
 payments/
bronze/
silver/
gold/

□ PHASE 3 — DATABRICKS CONNECTION TO ADLS

3.1 Create Service Principal

Method 1: Create Service Principal from Azure Portal

Step 1: Create an App Registration

1. Sign in to the Azure Portal:
   - Azure Portal
2. Search for **Microsoft Entra ID** (formerly Azure AD).
3. Select **App registrations**.
4. Click **New registration**.
5. Enter:
   - Name: `ADF-ServicePrincipal`
   - Supported account type: Single tenant (usually)
   - Redirect URI: Leave blank
6. Click **Register**.

Step 2: Get Application Details

After registration, note:

- Application (Client) ID
- Directory (Tenant) ID

These are required for authentication.

Example:

Client ID : xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Tenant ID : yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy

Step 3: Create Client Secret

1. Open the App Registration.
2. Go to:

Certificates & Secrets

3. Click:

New Client Secret

4. Enter:

Description: ADFSecret
Expiration: 24 months

5. Click **Add**.
6. Copy the Secret **Value** immediately.

Example:

Client Secret:
AbCdEf1234567890

 You cannot view the secret again after leaving the page.

Step 4: Assign Azure Role

Now give permissions to the Service Principal.

Go to the resource (Storage Account, Key Vault, Synapse, etc.)

Example:

Storage Account
→ Access Control (IAM)
→ Add Role Assignment

Select role:

Storage Blob Data Contributor

Assign access to:

User, Group or Service Principal

Search:

ADF-ServicePrincipal

Click:

Review + Assign

Azure → App Registration:

– DE_app

– new client-screte name - DE_ADFSecret

Copy:

- Client_id - xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
- Tenant_id - xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
- Secret or **value** - xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx -
- secretID- xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

3.2 Grant Permission

Assign role:

Storage Blob Data Contributor

The above service principal is not working . we have used manage identity unity catalog connector

Azure Databricks + ADLS Gen2 Integration using Unity Catalog (Managed Identity)

Objective

Connect Azure Databricks to Azure Data Lake Storage Gen2 (ADLS Gen2) using Unity Catalog, Managed Identity, Storage Credential, and External Location.

Environment Details

Storage Account

Storage Account Name:
vivekstorage01adls

Container:
raw

Files:

- customers.csv
- orders.csv
- products.csv
- payments.csv

Databricks Catalog

Catalog:
dedatabricsws

Schema:
demo

Step 1: Create Access Connector for Azure Databricks

Azure Portal:

Create Resource
→ Access Connector for Azure Databricks

Example:

Name:
adb-access-connector

Region:
Same region as Databricks Workspace

Step 2: Assign Storage Permissions

Azure Portal:

Storage Account
→ vivekstorage01adls
→ Access Control (IAM)
→ Add Role Assignment

Role:

Storage Blob Data Contributor

Assign To:

Databricks Access Connector Managed Identity

Wait 5-10 minutes for permission propagation.

Step 3: Create Storage Credential

Databricks:

Catalog Explorer
→ Create
→ Storage Credential

Configuration:

Credential Name:
vivek_sp_cred

Authentication Type:
Managed Identity

Access Connector ID:

Create Credential

Verification:

Storage Credential should be created successfully.

Step 4: Create External Location

Databricks:

Catalog Explorer
→ Create
→ External Location

Configuration:

Name:
raw_location

URL:
abfss://raw@vivekstorage01adls.dfs.core.windows.net/

Storage Credential:
vivek_sp_cred

Validation Results:

Success - Read
Success - List
Success - Write
Success - Delete
Success - Path Exists
Success - Hierarchical Namespace Enabled

File Events failures can be ignored.

Create External Location.

Step 5: Verify External Location

SQL:

SHOW EXTERNAL LOCATIONS;

Expected Output:

raw_location
abfss://raw@vivekstorage01adls.dfs.core.windows.net/

Step 6: Check Catalogs

SQL:

SHOW CATALOGS;

Output:

dedatabricsws
samples
system

Step 7: Use Catalog

SQL:

USE CATALOG dedatabricsws;

Step 8: Create Schema

SQL:

CREATE SCHEMA IF NOT EXISTS demo;

Verification:

SHOW SCHEMAS;

Expected:

demo

Step 9: Create External Volume

SQL:

CREATE EXTERNAL VOLUME dedatabricsws.demo.raw_volume
LOCATION 'abfss://raw@vivekstorage01adls.dfs.core.windows.net/';

Step 10: Verify Volume

SQL:

SHOW VOLUMES IN dedatabricsws.demo;

Expected:

Raw_volume

```
SHOW CATALOGS;
USE CATALOG dedatabricsws;

USE CATALOG dedatabricsws;

CREATE SCHEMA IF NOT EXISTS demo;

SHOW EXTERNAL LOCATIONS;

CREATE EXTERNAL VOLUME dedatabricsws.demo.raw_volume
LOCATION 'abfss://raw@vivekstorage01adls.dfs.core.windows.net/';

SHOW VOLUMES IN dedatabricsws.demo;
```

List Files from Volume

Notebook (PySpark):

display(
dbutils.fs.ls("/Volumes/dedatabricsws/demo/raw_volume")
)

```
display(
dbutils.fs.ls("/Volumes/dedatabricsws/demo/raw_volume")
)
df_customers = spark.read.option("header","true")\
    .csv("dbfs:/Volumes/dedatabricsws/demo/raw_volume/customers.csv")
display(df_customers)
```

Expected:

customers.csv
orders.csv
products.csv
payments.csv

Read CSV Files

Customers:

```
df_customers = spark.read
.option("header","true")
.csv("/Volumes/dedatabricsws/demo/raw_volume/customers.csv")

display(df_customers)
```

Products:

```
df_products = spark.read
.option("header","true")
.csv("/Volumes/dedatabricsws/demo/raw_volume/products.csv")

display(df_products)
```

Orders:

```
df_orders = spark.read
.option("header","true")
.csv("/Volumes/dedatabricsws/demo/raw_volume/orders.csv")

display(df_orders)
```

Payments:

```
df_payments = spark.read
.option("header","true")
.csv("/Volumes/dedatabricsws/demo/raw_volume/payments.csv")

display(df_payments)
```

Interview Explanation

Modern Databricks Architecture:

ADLS Gen2
↓
Managed Identity (Access Connector)
↓
Storage Credential
↓
External Location
↓
External Volume

↓
Delta Tables

Legacy Approach (Not Recommended):

- dbutils.fs.mount()
- Service Principal secrets in notebooks
- spark.conf.set()

Modern Unity Catalog Approach:

- Managed Identity
- Storage Credential
- External Location
- Volumes
- Unity Catalog Governance

```
SHOW CATALOGS;
USE CATALOG dedatabricsws;
USE CATALOG dedatabricsws;
CREATE SCHEMA IF NOT EXISTS demo;
SHOW EXTERNAL LOCATIONS;
CREATE EXTERNAL VOLUME dedatabricsws.demo.raw_volume
LOCATION 'abfss://raw@vivekstorage01adls.dfs.core.windows.net/';
SHOW VOLUMES IN dedatabricsws.demo;
--Create Bronze Database
CREATE DATABASE IF NOT EXISTS ecommerce_bronze;
show databases;
SELECT current_catalog(); --dedatabricsws
SELECT current_schema(); --default
-- If Unity Catalog is enabled:
--  NEVER use DATABASE
--  ALWAYS use SCHEMA + CATALOG
CREATE SCHEMA IF NOT EXISTS dedatabricsws.ecommerce_bronze;
CREATE SCHEMA IF NOT EXISTS dedatabricsws.ecommerce_silver;
CREATE SCHEMA IF NOT EXISTS dedatabricsws.ecommerce_gold;
-- in catalog dedatabricsws
show schemas ;
--ecommerce_bronze
--ecommerce_gold
--ecommerce_silver
```

Read all files from Volume (Bronze ingestion)

```
df_customers = spark.read.option("header", True).csv(
    "dbfs:/Volumes/dedatabricsws/demo/raw_volume/customers.csv"
)

# df_customers.show()
```

```
df_customers.write.mode("overwrite").saveAsTable("ecommerce_bronze.customers"
)
```

Read the order data and save in brone table

```
df_orders = spark.read.option("header", True).csv(
    "dbfs:/Volumes/dedatabricsws/demo/raw_volume/orders.csv"
)

df_orders.write.mode("overwrite").saveAsTable("ecommerce_bronze.orders")
```

Read the product data and save it.

```
df_products = spark.read.option("header", True).csv(
    "dbfs:/Volumes/dedatabricsws/demo/raw_volume/products.csv"
)

df_products.write.mode("overwrite").saveAsTable("ecommerce_bronze.products")
```

Read the payment data and save it.

```
df_payments = spark.read.option("header", True).csv(
    "dbfs:/Volumes/dedatabricsws/demo/raw_volume/payments.csv"
)

df_payments.write.mode("overwrite").saveAsTable("ecommerce_bronze.payments")
```

1 -ecommerce_bronze.customers
2 ecommerce_bronze.orders
3 ecommerce_bronze.products
4 ecommerce_bronze.payments

□ STEP 3: SILVER LAYER (CLEAN + TRANSFORM)

□ Goal

Clean nulls
Fix data types
Remove duplicates
Standardize schema

Customers Silver

```
from pyspark.sql.functions import col
df_orders = spark.table("dedatabricsws.ecommerce_bronze.customers")
df_orders.printSchema()
df_orders.show()
customers = spark.table("ecommerce_bronze.customers")

df_customers_silver = customers \
    .dropDuplicates() \
    .withColumn("customer_id", col("customer_id").cast("int"))

df_customers_silver.write.mode("overwrite").saveAsTable(
    "ecommerce_silver.customers"
)
df_orders = spark.table("dedatabricsws.ecommerce_silver.customers")
df_orders.printSchema()
df_orders.show()
```

Orders Silver

```
df_orders = spark.table("dedatabricsws.ecommerce_bronze.orders")
df_orders.printSchema()
df_orders.show()
```

```
orders = spark.table("ecommerce_bronze.orders")

df_orders_silver = orders \
    .dropDuplicates() \
    .withColumn("order_id", col("order_id").cast("int")) \
    .withColumn("total_amount", col("total_amount").cast("double"))

df_orders_silver.write.mode("overwrite").saveAsTable(
    "ecommerce_silver.orders"
)
df_orders = spark.table("dedatabricsws.ecommerce_silver.orders")
df_orders.printSchema()
df_orders.show()
```

Products Silver

```
products = spark.table("ecommerce_bronze.products")

df_products_silver = products.dropDuplicates()

df_products_silver.write.mode("overwrite").saveAsTable(
    "ecommerce_silver.products"
)
```

payments silver

```
df_orders = spark.table("dedatabricsws.ecommerce_bronze.payments")
df_orders.printSchema()
#df_orders.show()
payments = spark.table("ecommerce_bronze.payments")

df_payments_silver = payments \
    .dropDuplicates() \
    .withColumn("amount", col("amount").cast("double"))

df_payments_silver.write.mode("overwrite").saveAsTable(
    "ecommerce_silver.payments"
)
```

 RESULT SILVER LAYER
Clean datasets ready for analytics.

STEP 4: GOLD LAYER (BUSINESS AGGREGATIONS)

 Goal Create analytics-ready tables

4.1 Revenue by Customer

```
orders = spark.table("ecommerce_silver.orders")
payments = spark.table("ecommerce_silver.payments")

df_gold_customer_revenue = orders.join(
    payments,
    "order_id"
).groupBy("customer_id") \
 .sum("amount") \
 .withColumnRenamed("sum(amount)", "total_revenue")

df_gold_customer_revenue.write.mode("overwrite").saveAsTable(
    "ecommerce_gold.customer_revenue"
)
```

4.2 Product Sales

```
df_gold_product_sales = orders.groupBy("product_id") \
    .count() \
    .withColumnRenamed("count", "total_orders")

df_gold_product_sales.write.mode("overwrite").saveAsTable(
    "ecommerce gold.product sales"
)
```

4.3 Daily Revenue

```
from pyspark.sql.functions import to_date

df_daily_revenue = orders.join(payments, "order_id") \
    .withColumn("order_date", to_date("order_date")) \
    .groupBy("order_date") \
    .sum("amount") \
    .withColumnRenamed("sum(amount)", "daily_revenue")

df daily revenue.write.mode("overwrite").saveAsTable(
    "ecommerce_gold.daily_revenue"
)
```

□ GOLD RESULT

Now you have BI-ready tables:

1 - customer_revenue 2 - product_sales 3 - daily_revenu

STEP 5: LOAD GOLD → SNOWFLAKE

□ Goal Move curated data to Snowflake

Snowflake Connection (Databricks)

```
snowflake_options = {
  "host": "xxxxxx-xxxxx.snowflakecomputing.com",
  "port": 443,
  "sfUser": "snowDream",
  "sfPassword":  "xxxxxxxxxxxxxxx",
  "sfDatabase": "ECOMMERCE_DB",
  "sfSchema": "PUBLIC",
  "sfWarehouse": "COMPUTE_WH"
}
```

Write Gold Table to Snowflake

```
df_gold = spark.table("dedatabricsws.ecommerce_gold.customer_revenue")

df_gold.write \
  .format("snowflake") \
  .options(**snowflake_options) \
  .option("dbtable", "CUSTOMER_REVENUE") \
  .mode("overwrite") \
  .save()

df_gold = spark.table("dedatabricsws.ecommerce_gold.product_sales")

df_gold.write \
  .format("snowflake") \
  .options(**snowflake_options) \
  .option("dbtable", "PRODUCT SALES") \
  .mode("overwrite") \
```

```
  .save()

df_gold = spark.table("dedatabricsws.ecommerce_gold.daily_revenue")

df_gold.write \
  .format("snowflake") \
  .options(**snowflake_options) \
  .option("dbtable", "DAILY_REVENUE") \
  .mode("overwrite") \
  .save()
```

.

□ PHASE 8 — dbt MODELING

Step 2: Create dbt Cloud Account

Go to:

dbt Cloud Signup

Create account and workspace.

Step 3: Create GitHub Repository

Repository:

ecommerce-azure_snowflak_dbt

Upload:

dbt_project.yml

```
models/
├── staging/
├── marts/
└── schema.yml
```

Step 4: Connect GitHub to dbt Cloud

In dbt Cloud:

Create Project
↓

Repository Setup
↓
GitHub
↓
Authorize GitHub
↓
Select Repository

Selected:

ecommerce-azure_snowflak_dbt

Step 5: Configure Snowflake Connection

In dbt Cloud Environment:

Account : <snowflake_account>
User : <username>
Password : <password>
Role : ACCOUNTADMIN
Warehouse : COMPUTE_WH
Database : ECOMMERCE_DB
Schema : DBT_DEV

Test Connection:

Connection Successful

Step 6: Create Source Definition

File:

models/sources.yml

```
version: 2

sources:
 - name: ecommerce
   database: ECOMMERCE_DB
   schema: PUBLIC

   tables:
     - name: CUSTOMER_REVENUE
     - name: DAILY_REVENUE
     - name: PRODUCT_SALES
```

Step 7: Staging Models

stg_customer_revenue.sql

```
SELECT *
FROM {{ source('ecommerce', 'CUSTOMER_REVENUE') }}
```

stg_daily_revenue.sql

```
SELECT *
FROM {{ source('ecommerce', 'DAILY_REVENUE') }}
```

stg_product_sales.sql

```
SELECT *
FROM {{ source('ecommerce', 'PRODUCT_SALES') }}
```

Step 8: dbt Project Structure

```
ecommerce_dbt/
│
├── dbt_project.yml
│
├── models/
│   ├── sources.yml
│   │
│   ├── staging/
│   │   ├── stg_customer_revenue.sql
│   │   ├── stg_daily_revenue.sql
│   │   └── stg_product_sales.sql
│   │
│   ├── marts/
│   │   ├── dim_customer.sql
│   │   ├── dim_product.sql
│   │   ├── fct_revenue.sql
│   │   └── mart_dashboard.sql
│   │
│   └── schema.yml
│
└── README.md
```

==========
SNOWFLAKE
==========

```
use ECOMMERCE_DB;

select * from customer_revenue;

select * from DAILY_REVENUE;
```

```
select * from PRODUCT_SALES;

SELECT
   product_id,
   total_orders
FROM PRODUCT_SALES

SELECT
   order_date,
   daily_revenue
FROM DAILY_REVENUE

SHOW TABLES IN DATABASE ECOMMERCE_DB;

SHOW VIEWS IN SCHEMA DBT_DEV;

SHOW TABLES IN SCHEMA DBT_DEV;

SELECT CURRENT_DATABASE();
SELECT CURRENT_SCHEMA();

SHOW SCHEMAS IN DATABASE ECOMMERCE_DB;

SELECT
   table_schema,
   table_name,
   table_type
FROM ECOMMERCE_DB.INFORMATION_SCHEMA.TABLES
WHERE table_schema = 'DBT_DEV'
ORDER BY table_name;

SELECT COUNT(*) FROM ECOMMERCE_DB.PUBLIC.CUSTOMER_REVENUE;
SELECT COUNT(*) FROM ECOMMERCE_DB.DBT_DEV.STG_CUSTOMER_REVENUE;

SELECT *
FROM ECOMMERCE_DB.DBT_DEV.MART_DASHBOARD
LIMIT 20;
```

dbt Commands Successfully Used

1. Verify Configuration

dbt debug

Purpose:

Validate project
Validate Snowflake connection
Validate credentials

Result:

Success

---

2. Run Models

dbt run

Purpose:

Create views/tables in DBT_DEV schema

Result:

Success

---

3. Run Tests

dbt test

Purpose:

Validate not_null
Validate unique
Validate relationships

---

4. Build Entire Project

dbt build

Runs:

dbt run
dbt test

---

5. List Models

dbt ls

Purpose:

Display all models

---

6. Run Specific Model

dbt run --select mart_dashboard

---

7. Run Model with Dependencies

dbt run --select +mart_dashboard

---

8. Compile SQL

dbt compile

Purpose:

Generate final SQL without execution

---

9. Generate Documentation

dbt docs generate

Purpose:

Generate metadata and lineage

---

10. Full Refresh

dbt run --full-refresh

Purpose:

Drop and recreate all models

Final Interview Flow

ADLS Gen2
↓
Databricks
(Bronze → Silver → Gold)
↓
Snowflake
(PUBLIC)
↓
dbt Cloud
(DBT_DEV)
↓
Power BI

Interview Explanation

> Data is ingested into ADLS Gen2 and processed through Bronze, Silver, and Gold layers in Databricks. Curated Gold data is loaded into Snowflake. dbt Cloud is used

to build staging and mart models, manage dependencies through source definitions, perform testing, and generate lineage. The final mart tables are consumed by Power BI dashboards for analytics.

This is a complete end-to-end Azure + Databricks + Snowflake + dbt project suitable for Data Engineer interviews.

What dbt Created in Snowflake

```
ECOMMERCE_DB
│
├── PUBLIC                ← Source Tables
│   ├── CUSTOMER_REVENUE
│   ├── DAILY_REVENUE
│   └── PRODUCT_SALES
│
└── DBT_DEV               ← dbt Generated Objects
    │
    ├── STG_CUSTOMER_REVENUE  (VIEW)
    ├── STG_DAILY_REVENUE     (VIEW)
    ├── STG_PRODUCT_SALES     (VIEW)
    │
    ├── DIM_CUSTOMER          (TABLE)
    ├── DIM_PRODUCT           (TABLE)
    ├── FCT_REVENUE           (TABLE)
    └── MART_DASHBOARD        (TABLE)
```

```
SELECT

table_name,

table_type

FROM ECOMMERCE_DB.INFORMATION_SCHEMA.TABLES

WHERE table_schema='DBT_DEV'

ORDER BY table_name;
```

Why Staging Models Are Views

These came from:

```
models:
 ecommerce_dbt:
   staging:
     +materialized: view
```

So dbt created:

STG_CUSTOMER_REVENUE
STG_DAILY_REVENUE
STG_PRODUCT_SALES

as **views**.

Purpose:

- Lightweight
- No data duplication
- Always reads latest source data

Why Mart Models Are Tables

These came from:

```
models:
 ecommerce_dbt:
   marts:
     +materialized: table
```

So dbt created:

DIM_CUSTOMER
DIM_PRODUCT
FCT_REVENUE
MART_DASHBOARD

as **physical tables**.

Purpose:

- Faster reporting
- Better Power BI performance
- Store transformed business data

Verify Data Flow

Run:

```
SELECT COUNT(*) FROM ECOMMERCE_DB.PUBLIC.CUSTOMER_REVENUE;
SELECT COUNT(*) FROM ECOMMERCE_DB.DBT_DEV.STG_CUSTOMER_REVENUE;
```

Counts should match.

Check mart data:

```
SELECT *
FROM ECOMMERCE_DB.DBT_DEV.MART_DASHBOARD
LIMIT 20;
```

Verify Lineage

You now have:

```
CUSTOMER_REVENUE
      ↓
STG_CUSTOMER_REVENUE
      ↓
DIM_CUSTOMER
      ↓
FCT_REVENUE
      ↓
MART_DASHBOARD
```

This lineage is visible in dbt Cloud.

What to Say in Interviews

Why use Views in Staging?

> Staging models are materialized as views to avoid unnecessary storage and keep transformations lightweight.

Why use Tables in Mart Layer?

> Mart and fact models are materialized as tables for query performance and BI reporting.

Why separate schemas?

Source data remains untouched in PUBLIC schema, while dbt creates curated analytical objects in DBT_DEV schema.

What does dbt provide?

Source management, modular SQL, lineage tracking, testing, documentation, and CI/CD integration.

□ PHASE 9 — POWER BI

9.1 Connect Snowflake

- Server: Snowflake account
- Database: ECOM_DB
- Schema: MARTS

9.2 Build Model

Tables:

- SALES_FACT
- DIM_CUSTOMER
- DIM_PRODUCT

9.3 DAX Measures

Total Revenue = SUM(SALES_FACT[revenue])

9.4 Dashboard Pages

Page 1

- Revenue KPI

Page 2

- Sales Trend

Page 3

- Customer Insights

□ PHASE 10 — FINAL CHECKLIST (IMPORTANT)

Must Have

✔ ADLS raw ingestion
✔ Bronze/Silver/Gold layers
✔ Databricks transformation
✔ Snowflake warehouse
✔ dbt modeling
✔ Power BI dashboard

Without dbt

Your pipeline is:

ADLS
↓
Databricks Bronze
↓
Databricks Silver
↓
Databricks Gold
↓
Snowflake
↓
Power BI

This already works.

For small projects, you can stop here.

Then Why Was dbt Created?

dbt was created for teams where:

Data Engineers
Data Analysts
Analytics Engineers
Business Teams

all need to build transformations inside the warehouse.

What Happens in Real Companies?

Imagine:

Data Engineering Team

Loads data into Snowflake:

CUSTOMERS
ORDERS
PRODUCTS
PAYMENTS

or

CUSTOMER_REVENUE
PRODUCT_SALES
DAILY_REVENUE

Their job is done.

Analytics Team

Now asks:

Top Customers
Monthly Revenue
Customer Lifetime Value
Retention Analysis
Marketing Analytics

Without dbt:

SELECT ...

queries are scattered:

Power BI
Tableau
Excel
Ad-hoc SQL

Nobody knows:

- Which query is correct
- Which logic is latest
- Who changed what

This becomes a mess.

---

dbt Solves This

dbt brings:

Version Control
Testing
Documentation
Lineage
Modular SQL
CI/CD

In Your Project

You loaded:

CUSTOMER_REVENUE
PRODUCT_SALES
DAILY_REVENUE

into Snowflake.

dbt then created:

STG_CUSTOMER_REVENUE
STG_PRODUCT_SALES
STG_DAILY_REVENUE

Why?

Because source tables may contain:

duplicates
nulls

bad names
inconsistent types

Staging layer standardizes them.

Why Create DIM and FACT Tables Again?

Suppose tomorrow:

Marketing Team

needs:

Customer Segmentation

Finance Team

needs:

Monthly Revenue

Sales Team

needs:

Top Products

Instead of everybody writing SQL,

dbt creates:

DIM_CUSTOMER
DIM_PRODUCT
FCT_REVENUE

and everyone uses them.

Single source of truth.

What Did dbt Actually Do in Your POC?

Your source:

PUBLIC.CUSTOMER_REVENUE
PUBLIC.PRODUCT_SALES

PUBLIC.DAILY_REVENUE

dbt created:

DBT_DEV.STG_CUSTOMER_REVENUE
DBT_DEV.STG_PRODUCT_SALES
DBT_DEV.STG_DAILY_REVENUE

These are only views.

No data duplication.

Then dbt created:

DIM_CUSTOMER
DIM_PRODUCT
FCT_REVENUE
MART_DASHBOARD

These are business-ready datasets.

Power BI should ideally read:

MART_DASHBOARD

not raw tables.

Could We Have Built MART_DASHBOARD in Databricks?

Absolutely.

You could have done:

Databricks
↓
Gold Table
↓
Snowflake
↓
Power BI

and skipped dbt completely.

Many companies do this.

Then When Is dbt Valuable?

When:

Multiple teams use Snowflake

Finance
Marketing
Operations
Sales
Analytics

Need common business logic.

SQL Developers Work More Than Spark Developers

Many analysts know SQL.

Few know:

PySpark
Scala
Databricks internals

dbt lets them build models using SQL only.

Need Lineage

dbt automatically shows:

CUSTOMER_REVENUE
↓
STG_CUSTOMER_REVENUE
↓
DIM_CUSTOMER
↓
FCT_REVENUE
↓
MART_DASHBOARD

Without dbt, you manually track dependencies.

Need Testing

Example:

tests:
 - unique
 - not_null

dbt automatically validates data quality.

Databricks notebooks don't provide this framework out of the box.

Interview Answer

If asked:

> "Why use dbt when Databricks already transformed the data?"

Answer:

> Databricks is primarily used for large-scale data processing and ETL/ELT workloads. dbt is used to manage analytics transformations inside the data warehouse. It provides modular SQL development, testing, documentation, lineage, version control, and CI/CD capabilities. While Databricks can perform all transformations, dbt helps standardize and govern business logic for analysts and analytics engineers within Snowflake.

In Your Specific POC

ADLS
↓
Databricks
↓
Snowflake
↓
Power BI

would have been enough.

We added:

Snowflake
↓
dbt
↓
Power BI